CHAPTER 1

Property Kernels based on Vector Quantization

1. Introduction

Although there are notable exceptions [Dunker et al., 2001, Dunker and Obradovic, 2001], the structure and function of a protein are indelibly related; a protein's ability to induce chemical or physical change can be characterized, at some level, by illustrating what structural characteristics (or lack thereof) endow that protein with its particular abilities. While there are well established methods for experimentally resolving the structure of a protein (NMR, x-ray crystallography), these methods are costly, tedious, time consuming and rarely guarantee success. Because of these difficulties the rate of growth of the compendium of known protein structures is far outpaced by the growth of known sequences; growth largely driven by high-throughput sequencing efforts. In light of the important relationship between structure and function computational methods can potentially bridge the gap between the set of all known coding sequences and their associated protein structures.

The same is true when considering the function of a protein. Experimentally characterizing a protein's function is expensive and time consuming. Furthermore, with many organisms certain experiments, such as knockout studies in humans, are also unethical to conduct. *In-silico* methods also hold the greatest promise for functionally characterizing the large number of unannotated protein sequences that have been discovered.

Perhaps the simplest, and most intuitive manner in which structure and function can be inferred is through homology, or its common proxy sequence similarity. Traditionally protein sequences are represented as a string of letters representing amino acids. Alignment programs like BLAST [Altschul et al., 1990] do not explicitly consider the physiochemical properties associated with each amino acid, although it could be argued that substitution matrices like PAM [Dayhoff et al., 1978] and BLOSUM [Henikoff and Henikoff, 1992] implicitly encode the physiochemical similarities between amino acids.

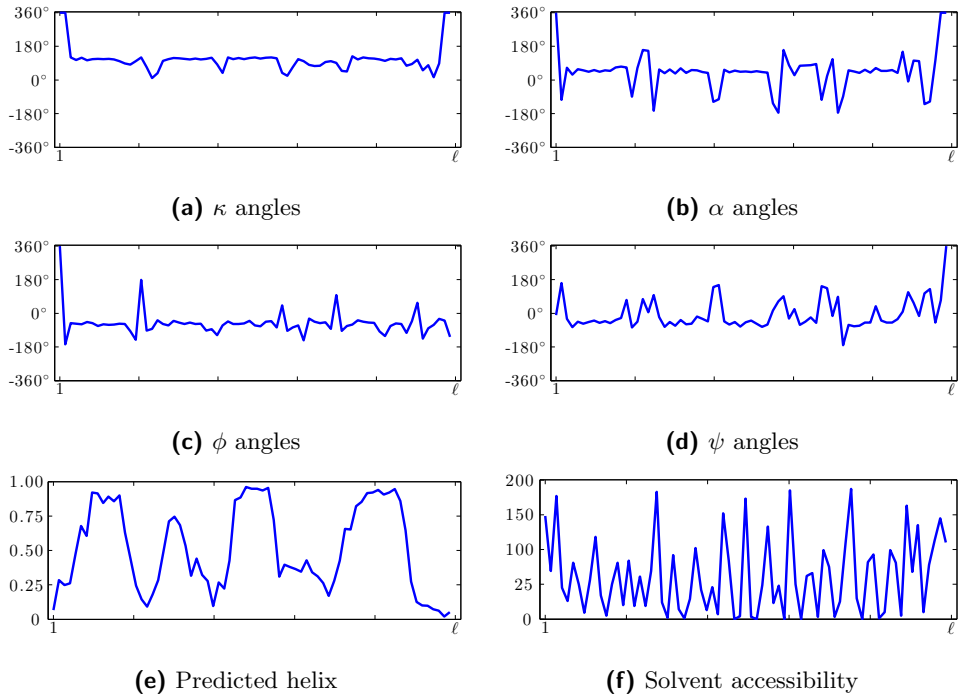
In this paper we have made a conscious choice to represent proteins, not as a sequence of letters or 3-D coordinates, but as a set of physiochemical and structural characteristics associated with each

amino acid. Although kernel methods can potentially be used to compare objects without explicitly representing them in feature space, we used a kernel function to encode each variable length property vector; which we refer to as a time series sequence, as a fixed length vector suitable for use in a supervised learning framework. Kernel have been widely used in computational biology [Schoölkopf et al., 2004], finding applications in the identification of functional residues [Vacic et al., 2010, Xin and Radivojac, 2011], the prediction of protein-protein interactions [Ben-Hur and Noble, 2005], the inference of function [Vert, 2002, De Bie et al., 2007], and the categorization of secondary strucutre [Leslie et al., 2002]. Our kernel function is novel in the way that we handle real valued, as opposed to discrete data, utilizing k-means clustering to discretize a continuous feature space.

While it may be beneficial to represent an amino acid as a set of real values based on physiochemical and structural properties; as opposed to a characters that don't explicitly encode any biological or chemical information, the shift from a representation that takes on an enumerable number of values to one that is uncountably infinite poses problems. For example, one common kernel representation of an object is the simple count kernel (detailed in Section 6.1). This representation has been used in the classification of text documents, protein sequences, as well as [insert tons of examples here]. A count kernel counts the number of times a single character, n-mer, or word appears in a sequence. Each sequence, or document, is then represented as a fixed length vector, whose length is defined by the cardinality of the set of possible n-length words that can be created by the relevant alphabet. For example, a count kernel using $n = 3$ length "mers" to represent a protein would result in each sequence being represented as a 20^3 length vector. Representing each amino acid, as a real value poses a problem because each amino acid can be assigned an infinitely uncountable set of values, resulting in a theoretically infinite feature vector when encoded using a count kernel.

We solve this problem in a novel way by discretizing n length vectors using k-means clustering (in this case we use m means). Each n length vector sampled from the larger series of data can then be discretized and represented by the centroid from the clustered data it is closest to. We chose to use k-means clustering as opposed to simply creating a lattice in n dimensional space, as done by Tuytelaars and Schmid [2007], because sampled property vectors do not fill the space evenly, but instead cluster around evolutionarily conserved or sterically preferred "spaces".

Finally, we compare the efficacy of explicitly using sequence properties to a sequence similarity based spectral kernel [Leslie et al., 2002] in the tasks of predicting protein function and SCOP fold.



(a) κ angles **(b)** α angles

(c) ϕ angles **(d)** ψ angles

(e) Predicted helix **(f)** Solvent accessibility

Figure 1.1. Time series representation of various protein properties. Make sure you note the protein used (number 1100 from SCOP data) and the length (74).

In order to simulate this we performed cross-validation on two distinct experiments in order to test the viability of the general VQ model; attempting to classify a protein according to its secondary structure or its molecular function. This was done for several types of structure and sequence property based features (Section 3.5) and for varied implementations of the basic VQ concept (Sections 6.1 to 6.4).

Our main goal is to introduce novel methodology for the non-alignment, kernel-based supervised classification of protein sequences and structures into distinct categories. We represent a protein as a set of time series properties such as hydrophobicity profiles, predictions of intrinsically disordered regions, or dihedral angle values if its structure is available. One way to augment the differing length time series signals for each sequence into fixed length features to be used as input for a given prediction method is to apply a vector quantization (VQ) kernel. VQ was initially developed as

a lossy method for compressing signals [Linde et al., 1980]. The basic idea behind this method (outlined in Figure 1.2) is to first break each signal up into overlapping fixed $n-$length segments (Figure 1.2a). These small segments are then clustered in n dimensional space, generating a set of m centroids representing each cluster (Figure 1.2b). Each segment can then be represented by which cluster it belongs to, and counts (or relative frequencies) that each subsegment for a property belongs to a cluster can be used to represent a sequence (Figure 1.2c). We evaluated methods on two distinct and relevant problems: (i) the classification of protein structures into structural classes and (ii) the prediction of protein function from amino acid sequence.

2. Methods

Let $\mathcal{S} = \{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, ...\}$ be a universe of protein sequences, where each $\mathbf{s} \in \mathcal{S}$ is a string of symbols from an alphabet of amino acids $\mathcal{A} = \{A, C, D, \ldots, Y\}$. Let also $\mathcal{S}_L \subset \mathcal{S}$ be a set of labeled sequences, e.g. those with known structural class or function, that is provided as training data. The objective is to use supervised framework to probabilistically annotate the remaining sequences, i.e. those from the unlabeled set $\mathcal{S}_U = \mathcal{S} - \mathcal{S}_L$.

To map protein sequences into a real-valued vector representation, let $\mathbf{s} = s_1 s_2 \ldots s_\ell$ be a length-ℓ protein sequence in $\mathcal{S}$ and $\mathbf{p} = (p_1, p_2, \ldots, p_\ell)$ some property vector defined by any particular mapping from $\mathcal{A}^\ell$ to $\mathbb{R}^\ell$. For example, $\mathbf{p}$ may be provided as a vector of hydrophobicity indices corresponding to amino acids in $\mathbf{s}$. Alternatively, it can be represented as predicted helical propensities as outputted by some predictor of secondary structure. For those sequences with available structures, $\mathbf{p}$ may correspond to a sequence of dihedral angles calculated from the protein structure model of $\mathbf{s}$.

Consider now a single property vector $\mathbf{p}$, such as hydrophobicity profile, corresponding to a particular sequence $\mathbf{s} \in \mathcal{S}$. We split $\mathbf{p}$ into n-dimensional sub-vectors $\mathbf{p}_{[1,n]}$, $\mathbf{p}_{[2,n+1]}$, $\ldots$, $\mathbf{p}_{[\ell-n+1,\ell]}$, where $\mathbf{p}_{[i,i+j]} = (p_i, p_{i+1}, ..., p_{i+j})$, and $n \ll \ell$ is a small integer. For example, $\mathbf{p}_{[1,n]}$ corresponds to the first n elements of $\mathbf{p}$ as shown in Figure 1.2(a). For a property vector (amino acid sequence) of length ℓ, there are $\ell - n + 1$ length-n sub-vectors.

For a single property type and property vector $\mathbf{p}$ corresponding to a sequence $\mathbf{s} \in \mathcal{S}$, $\mathbf{p}$ can be split into n-dimensional sub-vectors $\mathbf{p}_{[1,n]}$,$\mathbf{p}_{[2,n+1]}$,$\ldots$, $\mathbf{p}_{[\ell-n+1,\ell]}$, where $\mathbf{p}_{[i,i+j]} = (p_i, p_{i+1}, ..., p_{i+j})$, and $n \ll \ell$ is a small integer. For example, $\mathbf{p}_{[1,n]}$ corresponds to the first n elements of $\mathbf{p}$ as shown in

Figure 1.2(a). For a property vector (amino acid sequence) of length ℓ, there are $\ell - n + 1$ length-n sub-vectors.

Given a set of length-n property sub-vectors $\mathcal{P}$ derived from the sequence universe $\mathcal{S}$, we generate a partition of $\mathbb{R}^n$ into m regions $\mathcal{R} = \{R_1, R_2, \ldots, R_m\}$. These regions are represented by a set of n-dimensional vectors, or centroids, $\mathcal{C} = \{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_m\}$. Each region R_i represents a Voronoi region such that

$$R_i = \{\mathbf{x} : d(\mathbf{x}, \mathbf{c}_i) \leq d(\mathbf{x}, \mathbf{c}_j), j \neq i\},$$

where $d(\mathbf{x}, \mathbf{c})$ is the Euclidean distance between vector $\mathbf{x}$ and centroid $\mathbf{c}$. We determine $\mathcal{C}$ using k-means clustering, where the initial set of clusters is generated by the splitting method [Gersho and Gray, 1992].

2.1. Count-based property kernels. Variable length property vectors can be transformed into fixed m-length vectors using the partition of $\mathbb{R}^n$ defined by $\mathcal{R}$. Specifically, a property vector $\mathbf{p}$ is mapped into a vector of length m as

$$\mathbf{x} = (\varphi_1(\mathbf{p}), \varphi_2(\mathbf{p}), \ldots, \varphi_m(\mathbf{p})),$$

where $\varphi_i(\mathbf{p})$ is the number of n-dimensional vectors $\mathbf{p}_{[.]}$ in $\mathbf{p}$ that belong to partition R_i. Given two property vectors $\mathbf{p}$ and $\mathbf{q}$ and their respective count vectors $\mathbf{x}$ and $\mathbf{y}$, a *count-based property kernel* is defined as

$$k(\mathbf{p}, \mathbf{q}) = \mathbf{x}^T \mathbf{y},$$

where T is the transpose operator. Note that in this notation each property vector is assumed to be a column vector, i.e. $(a, b, c) = [a\ b\ c]^T$, as in Strang [2003]. Since the function $k(\mathbf{p}, \mathbf{q})$ is defined as an inner product between two count vectors, it is a kernel function [Haussler, 1999].

The count-based property kernels can be generalized by utilizing similarities between centroids to augment partial counts to each of the regions in $\mathcal{R}$. First, a similarity function between any two centroids is defined as

$$s(\mathbf{c}_i, \mathbf{c}_j) = e^{-\alpha \cdot \frac{d(\mathbf{c}_i, \mathbf{c}_j)}{d_{\max}}},$$

where $d(\mathbf{c}_i, \mathbf{c}_j)$ is the Euclidean distance between $\mathbf{c}_i$ and $\mathbf{c}_j$ and $d_{\max}$ is the maximum distance between any two centroids in $\mathcal{C}$. The parameter α is a positive constant that defines an extent to which large distances are penalized by the non-linear mapping above, and is optimized in the model evaluation step. A generalized property kernel can now be defined as

$$k_g(\mathbf{p}, \mathbf{q}) = (\mathbf{S}\mathbf{x})^T(\mathbf{S}\mathbf{y}) = \mathbf{x}^T\mathbf{S}^T\mathbf{S}\mathbf{y} = \mathbf{x}^T\mathbf{Q}\mathbf{y},$$

where elements of the similarity matrix $\mathbf{S}$ are defined as $s_{ij} = s(\mathbf{c}_i, \mathbf{c}_j)$. Clearly, for $\mathbf{S} = \mathbf{I}$, i.e. $\alpha \to \infty$, the generalized kernel is equivalent to the basic count-based property kernel. If $\mathbf{S}$ is a non-singular matrix, it is easy to show that $\mathbf{Q}$ is a positive definite matrix [Strang, 2003]. Therefore, $k_g(\mathbf{p}, \mathbf{q})$ is a kernel function. Note that the generalized count-based kernel does not lead to a sparse representation, even for large m.

2.2. Additive property kernels. Let us again consider vector $\mathbf{p}$ and partition $\mathcal{R}$. We map $\mathbf{p}$ into an m-dimensional as

$$\mathbf{x} = (\psi_1(\mathbf{p}), \psi_2(\mathbf{p}), \ldots, \psi_m(\mathbf{p})),$$

where

$$\psi_i(\mathbf{p}) = \sum_{j=1}^{\ell-n+1} s\left(\mathbf{p}_{[j,j+n-1]}, \mathbf{c}_i\right),$$

is the cumulative similarity between vectors $\mathbf{p}_{[.]}$ in $\mathbf{p}$ and centroid $\mathbf{c}_i$, and $s(\cdot)$ is the vector similarity function defined above. Given two property vectors $\mathbf{p}$ and $\mathbf{q}$ and their respective count vectors $\mathbf{x}$ and $\mathbf{y}$, we now define an *additive property kernel* as $k(\mathbf{p}, \mathbf{q}) = \mathbf{x}^T\mathbf{y}$.

2.3. Combined kernel function. Given a set of property kernels $\{k_i(\mathbf{x}, \mathbf{y})\}$, we construct the composite property kernel as a linear combination

$$k(\mathbf{x}, \mathbf{y}) = \sum_i k_i(\mathbf{x}, \mathbf{y})$$

where prior to combining, each kernel is normalized using

$$k(\mathbf{x}, \mathbf{y}) \leftarrow \frac{k(\mathbf{x}, \mathbf{y})}{\sqrt{k(\mathbf{x}, \mathbf{x})k(\mathbf{y}, \mathbf{y})}}.$$

2.4. Computational complexity. The computation of the count vector can be accomplished in $O(\ell mn)$ time if each n-dimensional vector from $\mathbf{p}$ is compared with all centroids in $\mathcal{C}$. Approximation algorithms are available through a decision tree like organization of the centroids. In such a case, only $\log m$ operations are needed resulting in $O(\ell n \log m)$ time [Gersho and Gray, 1992]; however, there is no guarantee that the closest centroid will be found. Using a full-search scheme, the basic and generalized property kernels can be computed in $O(\ell nm)$ and $O(m^2 + \ell nm)$ times, respectively. The storage requirements include $O(mn)$ for storing $\mathcal{C}$ and $O(m^2)$ for storing $\mathbf{Q}$. The additive kernel has complexity $O(\ell nm)$.

3. String kernel

As a baseline method with which we could compare results from our VQ model we utilized a string kernel as described by Leslie et al. [2002] for a wide range of word sizes $(1 \ldots 10)$. For a given word size l, a 20^l-length vector was created for a protein, where each value represented the number of times a potential l length substring generated using the 20 amino acid alphabet occurred. An ℓ-length sequence, $\mathbf{s}$, would contain $\ell - l + 1$ such overlapping strings.

4. Experiments and data

We utilized Astral 1.75A, which represents a database of proteins classified according to secondary structures as defined by SCOP [Murzin et al., 1995]. In order to ensure that redundancy in the data set did not lead to inflated assessment of performance we chose the 40% non-redundant version of the database. Prediction was performed as a one-versus-all classification at the SCOP fold level for proteins categorized as α, β, $\alpha + \beta$, or α/β folds. Table 1.1 summarizes the positive and negative data points used for each category of SCOP fold.

In the second experiment we attempted to distinguish enzymes, or those proteins annotated with the term "catalytic activity", from all other proteins. GO annotations were obtained from the April 2012 release of Swiss-Prot [**?**] in conjunction with the May 4, 2012 version of the gene ontology [Ashburner et al., 2000]. Only annotations supported by evidence codes EXP, IDA, IPI, IMP, IGI, IEP, TAS, IC were used. This resulted in a total of $24,882$ proteins with experimentally verified annotations, $9,506$ of which were annotated with the term "catalytic activity" and $18,936$ which represented putative negative data points.

SCOP Fold	positives	negatives
all α	$1,901$	$7,486$
all β	$2,175$	$7,212$
$\alpha+\beta$	$2,665$	$6,722$
α/β	$2,646$	$6,741$

Table 1.1. Summary of data used for SCOP fold classification documenting the number of positives and negatives used for the classification of protein structures as α, β, $\alpha+\beta$, or α/β folds.

4.1. Cross validation. We performed 10-fold cross validation on both data sets. For each fold non-overlapping test and training sets, consisting of $\frac{1}{10}$-ths and $\frac{9}{10}$-ths of the data respectively, were generated. All 10 testing sets were mutually exclusive and collectively exhaustive. We created a single set of testing folds for all classification sub-tasks: one set for SCOP fold classification and a separate for the prediction of catalytic activity.

We took steps in order to ensure that positive data points for each class were evenly distributed among each fold. Because protein structures are only assigned to a single SCOP class ensuring equal distribution among testing folds was very simple: we merely assigned a random $\frac{1}{10}$ of all proteins for a tested SCOP class to each testing fold. The same was done when creating testing folds for "catalytic activity" classification. We equally distributed terms belonging to each subclass of "catalytic activity" ensuring that proteins that may have belonged to more than one class only appear in one testing fold and did not appear more than once in each fold.

4.2. AUC. For each binary classification experiment we calculated the area under the receiver operating characteristic (AUC) . In order to do this predictor scores were first normalized to fall between the range of $[0,1]$. A decision threshold, τ, was varied from 0 to 1 in increments of .01. At each threshold sensitivity (sn_τ) and specificity (sp_τ) were calculated. The AUC of a model was then calculated using the trapezoid rule as

$$AUC = \sum_{\tau\in\mathcal{T}} sp_\tau - sp_{(\tau-.01)} \frac{sn_\tau - sn_{(\tau-.01)}}{2}$$

where $\mathcal{T}$ defines the set of thresholds, τ ranging from 0 to 1 in increments of .01.

4.3. Weighted AUC. While we evaluated each feature type for a combination of window size and number of clusters on each classification task separately we also desired to obtain a single value that could be used to benchmark each combination of parameters on all evaluated SCOP classes. In order to do this we averaged AUC across multiple one-vs-all classification tasks. While it would be simple to merely average the AUC values for each SCOP fold; if, for example, a fold class has few positive examples relative to others it will disproportionately influence the average.

In order to account for class imbalances between each SCOP fold we utilized a weighted method for calculating AUC: $AUC^{\mathbf{w}}$. If we have a set of classes $C = \{c_1, c_2..c_n\}$ on which binary-predictions were carried out, with associated counts of positive data-points $N = \{n_1, n_2...n_n\}$ and associated AUC values $AUC = \{auc_1, auc_2...auc_n\}$ we calculate the weighted average AUC, as

$$AUC^{\mathbf{w}} = \frac{\sum\limits_{i=1..n} n_i \times auc_i}{\sum\limits_{i=1..n} n_i}$$

4.4. Signal-to-noise ratio. We also calculated the signal-to-noise ration (SNR) obtained when encoding and decoding property based representations of proteins using VQ. Given an original property vector $\mathbf{p}$ and the encoded, the reconstructed version of this vector $\hat{\mathbf{p}}$ the signal to noise ratio was calculated using the logarithmic decibel scale as

$$SNR_{dB}(\mathbf{p}, \hat{\mathbf{p}}) = \log_{10}\left(\frac{\sum\limits_{i=1}^{\ell} p_i^2}{\sum\limits_{i=1}^{\ell} (p_i - \hat{p}_i)^2}\right)$$

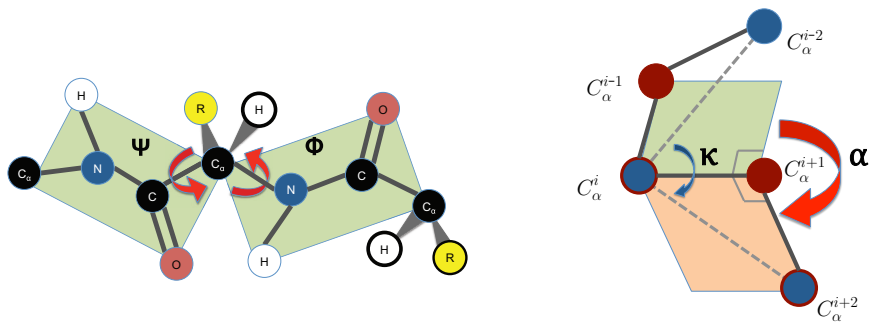
On this scale one decibel signifies that the noise (or squared difference between the original and reconstructed signal) represents $^1/_{10}$-th of the signal. SNR represents the amount of information lost when a property is encoded and decoded using VQ.

5. Mapping proteins into property vectors

Let $\mathbf{s} = s_1, s_2 \ldots s_\ell$ be a length-ℓ protein sequence in $\mathcal{S}$ and $\mathbf{p} = (p_1, p_2, \ldots, p_\ell)$ some property vector corresponding to $\mathbf{s}$. Vector $\mathbf{p}$ can be defined by any mapping of $\mathcal{A}^\ell$ to $\mathbb{R}^\ell$. For example, $\mathbf{p}$ may be provided as a vector of hydrophobicity indices corresponding to amino acids in $\mathbf{s}$. Alternatively, it can be represented as predicted helical propensities as outputted by some predictor of secondary

structure. For those sequences with available structures, $\mathbf{p}$ may correspond to a sequence of dihedral angles calculated from the protein structure model of $\mathbf{s}$.

5.1. Structure based properties. Structure based properties were generated by converting the 3D coordinates of amino acids into backbone angles. These angles can be used to position each amino acid in relation to surrounding atoms in the protein chain. The usefulness of representing a structure in this manner is that backbone angle are the same regardless of how the overall protein is oriented in space, whereas 3D coordinates can change for the exact same structure simply by rotating or shifting it.



(a) ϕ and ψ tortion angles

(b) κ and α angles

Figure 1.2. The different types of angles that can be measured from protein structures are shown. **??** illustrates the backbone torsion angles, ϕ and ψ, for a C_α atom that can be measured between its adjacent nitrogen and C' atoms respectively. **??** illustrates the angles that can be measures between successive C_α atoms: C_α^{i-1} and C_α^{i+2} in the case of κ angles; and C_α^{i-2} and C_α^{i+2} in the case of α angles.

We utilized four types of backbone angles: κ, α, ϕ, and ψ. Figure 1.3 illustrates how these backbone angles are defined using an protein backbone chain. As illustrated by **??**, ϕ and ψ angles represent the torsion angle, or amount of rotation, of the bond between C_α side-chain attached carbon atom and the adjacent nitrogen (ϕ angle) and C' carbon (ψ angle) atoms. While ϕ and ψ represent torsion angles between adjacent atoms in a protein's backbone, κ and α angles represent the relationship between more distant C_α atoms. An ℓ amino acid length sequence, $\mathbf{s} = s_1, s_2 \ldots s_\ell$, will have ℓ associate C_α carbon atoms: $C_\alpha^1, C_\alpha^2 \ldots C_\alpha^\ell$. Using **??** as a reference, if one designates a C_α^i atom as a vertex (in the angular sense), the κ angle represents the angle obtained by extending a

line from the C_α^i atom's position to the C_α^{i-2} and C_α^{i+2} atoms. The α angle represents the dihedral angle measured between the C_α^{i-1} and C_α^{i+2} atoms when two planes are generated using the C_α^i atom to the C_α^{i+1} to define the line where the two planes meet.

κ, α, ϕ, and ψ angles were obtained using DSSP [Kabsch and Sander, 1983] to convert PDB structures into backbone angles. In addition to generating backbone angles, DSSP also output solvent accessibility values which we used as a fifth "structure" based feature.

5.2. Sequence based properties. We generated seven sequence property based properties for both the task of categorizing structures into SCOP folds and predicting GO function. These features were generated in order to represent a biologically relevant property associated with a region of a protein sequence.

(i) hydrophobicity, calculated using the the Kyte-Doolittle scale [Kyte and Doolittle, 1982] in a sliding window with $w = 11$, (ii) flexibility, calculated as predicted B-factors using our previous model [Smith et al., 2003], secondary structure predictions of (iii) helix, (iv) loop and (v) sheet propensities using our in-house predictor, and intrinsic disorder, (vi) using the previously published VSL2B model [Peng et al., 2006] as well as (vii) using PDBDisorder XXX? .

5.2.1. *Tested values of n (window size) and m (number of centroids).* Not sure where to put this, but we need to list the tested values of m and n somewhere. Might fit better in methods? We tested a range of combinations of values for n (window size), and m (number of centroids) for each feature. For values of n we tested $n = \{2^i : i = 2 \ldots 5\}$. Similarly, for the number of centroids, m, we tested $m = \{2^i : i = 6, 8, 10, 12\}$. For all values of m and n we performed k-means clustering using 10^6 randomly sampled vectors from all data points.

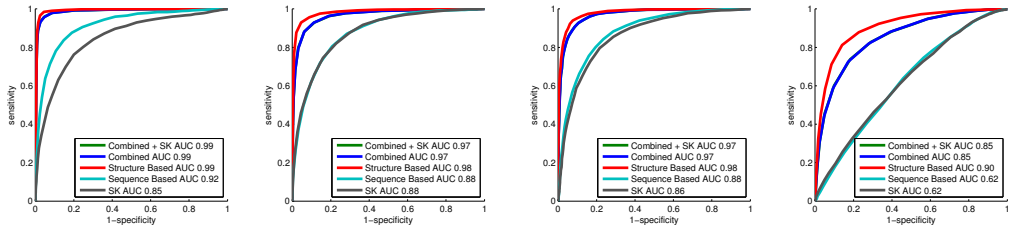
6. Results

6.1. SCOP fold. Table 1.2 shows the performance of each property for when predicting SCOP folds. Among structure based properties we found that κ backbone angles had the best performance, both for individual SCOP folds and in terms of its $AUC^{\mathbf{w}}$ (0.961). Solvent accessibility (SolAcc) values performed the worst out of structure based properties, obtaining the lowest AUC among structure properties for all SCOP folds and in terms of $AUC^{\mathbf{w}}$ (0.868). In general all structure based properties outperformed sequence based properties. Amongst sequence based properties predicted secondary structures performed the best, especially the predictions that a residue is in a helix

($AUC^{\mathbf{w}} = 0.788$), a loop ($AUC^{\mathbf{w}} = 0.771$), or a sheet ($AUC^{\mathbf{w}} = 0.787$). Calculated hydrophobicity and VSL2B based predictions of disorder propensity performed the worst ($AUC^{\mathbf{w}} = 0.707$ and $AUC^{\mathbf{w}} = 0.682$ respectively). As a predictor of disorder PDBDisorder based features performed better than VSL2B ($AUC^{\mathbf{w}} = 0.764$), although this may be based on information leak due to this predictor being trained on PDB data.

Property\Category	All α			All β			α/β			$\alpha+\beta$			$AUC^{\mathbf{w}}$		
	m	n	AUC	m	n	AUC	m	n	AUC	m	n	AUC	m	n	$AUC^{\mathbf{w}}$
Alpha	4,096	8	0.994	4,096	8	0.982	4,096	16	0.967	4,096	32	0.829	8	4,096	0.930
Kappa	4,096	16	0.995	4,096	16	0.986	4,096	16	0.978	4,096	32	0.903	32	4,096	0.961
Phi	4,096	16	0.990	4,096	16	0.975	4,096	16	0.964	4,096	32	0.809	32	4,096	0.920
Psi	4,096	8	0.991	4,096	16	0.981	4,096	16	0.970	4,096	32	0.850	32	4,096	0.938
SolAcc	4,096	8	0.960	4,096	8	0.951	4,096	32	0.914	4,096	32	0.710	32	4,096	0.868
BFactor	256	8	0.790	256	8	0.709	64	8	0.809	16	4	0.587	4	64	0.717
Helix	16	4	0.868	16	8	0.871	64	16	0.842	64	32	0.637	32	64	0.788
Hydro	256	4	0.711	1,024	4	0.728	64	2	0.834	64	2	0.585	4	256	0.707
Loop	256	16	0.872	64	8	0.848	16	32	0.821	1,024	32	0.607	16	64	0.771
PDBDisorder	1,024	8	0.842	64	4	0.849	64	4	0.819	256	32	0.591	4	64	0.764
Sheet	64	2	0.904	64	8	0.853	256	16	0.836	64	32	0.641	32	64	0.787
VSL2B	16	2	0.699	64	32	0.628	64	8	0.812	64	2	0.589	8	64	0.682
String Kernel	–	2	0.863	–	3	0.878	–	4	0.860	–	5	0.634	–	4	0.794
Sequence	64	16	0.915	64	16	0.876	64	16	0.880	64	16	0.621	64	16	0.813
Structure	4,096	32	0.992	4,096	32	0.983	4,096	32	0.979	4,096	32	0.903	4,096	32	0.961
Sequence + Structure	–	–	0.989	–	–	0.967	–	–	0.970	–	–	0.851	–	–	0.939
Sequence + Structure + SK	–	–	0.989	–	–	0.967	–	–	0.970	–	–	0.851	–	–	0.939

Table 1.2. Optimal performance, according to AUC, of each property based feature when predicting SCOP fold. For each property feature the combination of m and n values that obtained the highest AUC for an individual SCOP fold are reported. The last three columns show the weighted AUC, $AUC^{\mathbf{w}}$, obtained across all SCOP folds. Results when combining all sequence and property based features are shown in the bottom section of the table. Because different combinations of m and n are used when combining properties these values are not shown for the Sequence + Structure and Sequence + Structure + SK combination of features.



(a) Alll α **(b)** All β **(c)** $\alpha/\ \beta$ **(d)** $\alpha + \beta$

Figure 1.3. SCOP ROC Curves showing the performance of each combination of combined features and the string kernel when predicting each category of SCOP fold tested.

6.1.1. *"Catalytic activity".* The first column of results in Table 1.2 show the performance of each property when predicting whether a protein is annotated with the "catalytic activity" term. We found that disorder based predictions (VSL2B $AUC = 0.742$ and PDBDisorder $AUC = 0.718$) and predicted BFactors ($AUC = 0.722$) performed the best, whereas, contrary to their performance in distinguishing between SCOP folds, predicted secondary structures performed the worst (Helix $AUC = 0.687$, Loo $AUC = 0.698$, and Sheet $AUC = 0.681$).

6.2. Combined kernel performance.

6.2.1. *SCOP fold.*

6.2.2. *"Catalytic activity".* Don't forget to insert catalytic activity ROC curve here

7. Discussion

Traditionally protein sequences are represented as a string of letters representing amino acids, or as the coordinates of atoms in 3-dimensional space. Alignment algorithms like BLAST [Altschul et al., 1990] do not explicitly consider the physiochemical properties associated with each amino acid; it would be a stretch to argued that through substitution matrices like PAM [Dayhoff et al., 1978] and BLOSUM [Henikoff and Henikoff, 1992] alignment algorithms implicitly encode physiochemical similarities between amino acids. In this paper we making an conscious choice to encode proteins; not as a sequence of letters, or 3-D coordinates, but as explicitly as the physiochemical and structural characteristics associated with those amino acids. As previously stated, this is not a wholly novel idea; although, to the best of our knowledge this is the first time that

8. Supplementary data

Bibliography

S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.

A. Andreeva, D. Howorth, S. E. Brenner, T. J. P. Hubbard, C. Chothia, and A. G. Murzin. Scop database in 2004: refinements integrate structure and sequence family data. *Nucleic Acids Research*, 32(suppl 1):D226–D229, 2004.

M. Ashburner et al. Gene ontology: tool for the unification of biology. The Gene Ontology Consortium. *Nature Genetics*, 25(1):25–29, 2000.

A. Ben-Hur and W. S. Noble. Kernel methods for predicting protein-protein interactions. *Bioinformatics*, 21(1):i38–i46, 2005.

T. U. Consortium. Reorganizing the protein space at the universal protein resource (uniprot). *Nucleic Acids Research*, 40(Database issue):D71–5, 2012. ISSN 13624962.

M. O. Dayhoff, R. M. Schwartz, and B. C. Orcutt. A model of evolutionary change in proteins. *Atlas of protein sequence and structure*, 5(Suppl 3):345–352, 1978.

T. De Bie, L.-C. T. Tranchevent, L. Van Oeffelen, and Y. Moreau. Kernel-based data fusion for gene prioritization. *Bioinformatics*, 23(13):i125–i132, 2007.

A. K. Dunker and Z. Obradovic. *Nature Biotechnology*, 19(9):805–6, 2001.

A. K. Dunker, J. D. Lawson, C. J. Brown, R. M. Williams, P. Romero, J. S. Oh, C. J. Oldfield, A. M. Campen, C. M. Ratliff, K. W. Hipps, J. Ausio, M. S. Nissen, R. Reeves, C. Kang, C. R. Kissinger, R. W. Bailey, M. D. Griswold, W. Chiu, E. C. Garner, and Z. Obradovic. Intrinsically disordered proteins. *Journal of Molecular Graphics and Modelling*, 19(1):26–59, 2001.

A. Gersho and R. M. Gray. *Vector quantization and signal compression*, volume 159. Springer, 1992.

D. Haussler. Convolution kernels on discrete structures. Technical Report 1, University of California at Santa Cruz, 1999.

S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences of the United States of America*, 89(22):10915–10919, 1992.

W. Kabsch and C. Sander. Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, 22(12):2577–2637, 1983.

J. Kyte and R. F. Doolittle. A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1):105–132, 1982.

C. Leslie, E. Eskin, and W. S. Noble. The spectrum kernel: a string kernel for SVM protein classification. *Pacific Symposium On Biocomputing*, 575(50):564–575, 2002.

Y. Linde, A. Buzo, and R. Gray. An algorithm for vector quantizer design. *IEEE Transactions on Communications*, 28(1):84–95, 1980. ISSN 00962244.

A. G. Murzin, S. E. Brenner, T. Hubbard, C. Chothia, and Others. Scop: a structural classification of proteins database for the investigation of sequences and structures. *Journal of molecular biology*, 247(4):536–540, 1995.

K. Peng, P. Radivojac, S. Vucetic, A. K. Dunker, and Z. Obradovic. Length-dependent prediction of protein intrinsic disorder. *BMC Bioinformatics*, 7(1):208, 2006.

B. Schoölkopf, K. Tsuda, and J.-P. Vert. *Kernel methods in computational biology*. The MIT press, 2004.

D. K. Smith, P. Radivojac, Z. Obradovic, A. K. Dunker, and G. Zhu. Improved amino acid flexibility parameters. *Protein Science*, 12(5):1060–1072, 2003.

G. Strang. *Introduction to Linear Algebra*. Wellsley-Cambrige Press, 2003. ISBN 9780961408893.

T. Tuytelaars and C. Schmid. Vector Quantizing Feature Space with a Regular Lattice. In *2007 IEEE 11th International Conference on Computer Vision*, pages 1–8. IEEE, 2007. ISBN 978-1-4244-1630-1.

V. Vacic, L. M. Iakoucheva, S. Lonardi, and P. Radivojac. Graphlet kernels for prediction of functional residues in protein structures. *Journal of computational biology a journal of computational molecular cell biology*, 17(1):55–72, 2010.

J.-P. Vert. A tree kernel to analyse phylogenetic profiles. *Bioinformatics*, 18(1):S276–S284, 2002.

F. Xin and P. Radivojac. Computational methods for identification of functional residues in protein structures. *Current protein peptide science*, 120(6):456–69, 2011.